

India Air Quality & Crop Yield — Lab 1 & 2

Data Preprocessing, Visualisation & Exploration

Combined Lab Sheet — June 2025

June is globally recognised as Environment Awareness Month, with World Environment Day celebrated on June 5. This lab investigates whether worsening air quality across Indian states is linked to declining agricultural output.

Datasets:

- `city_day.csv` — City-level AQI readings (2015–2020), 26 cities, 29,531 rows
- `crop_production.csv` — State-level crop production records (1997–2015), 246,091 rows

Setup — Imports & Configuration

```
In [42... import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import seaborn as sns
import warnings

warnings.filterwarnings('ignore')

# Plotting style
plt.rcParams.update({
    'figure.dpi': 120,
    'figure.facecolor': 'white',
    'axes.spines.top': False,
    'axes.spines.right': False,
    'font.family': 'DejaVu Sans'
})
sns.set_palette('muted')

print('Libraries loaded successfully.')
```

Libraries loaded successfully.

Lab 1 — Data Preprocessing and Visualisation

Task 1 — First-Impression Data Profile

Reasoning

Before trusting any dataset, a data scientist needs to understand: shape, column types, missing value counts, value ranges (via `describe()`), and unique identifiers. We profile both files systematically before touching a single row.

```
In [43... # Load both datasets
aqi = pd.read_csv('city_day.csv')
crop = pd.read_csv('crop_production.csv')

print('=' * 60)
print('FILE 1 : city_day.csv')
print('=' * 60)
print(f'Shape           : {aqi.shape[0]:,} rows × {aqi.shape[1]} c
print(f'Columns        : {list(aqi.columns)}')
print(f'Date range      : {aqi["Date"].min()} → {aqi["Date"].max()}
print(f'Unique cities   : {aqi["City"].nunique()} → {sorted(aqi["C
print('\nNull counts:')
print(aqi.isnull().sum())
print('\nDescriptive statistics:')
display(aqi.describe())
```

=====

FILE 1 : city_day.csv

=====

Shape : 29,531 rows × 16 columns

Columns : ['City', 'Date', 'PM2.5', 'PM10', 'NO', 'NO2', 'NOx', 'NH3', 'CO', 'SO2', 'O3', 'Benzene', 'Toluene', 'Xylene', 'AQI', 'AQI_Bucket']

Date range : 2015-01-01 → 2020-07-01

Unique cities : 26 → ['Ahmedabad', 'Aizawl', 'Amaravati', 'Amritsar', 'Bengaluru', 'Bhopal', 'Brajrajnagar', 'Chandigarh', 'Chennai', 'Coimbatore', 'Delhi', 'Ernakulam', 'Gurugram', 'Guwahati', 'Hyderabad', 'Jaipur', 'Jorapokhar', 'Kochi', 'Kolkata', 'Lucknow', 'Mumbai', 'Patna', 'Shillong', 'Talcher', 'Thiruvananthapuram', 'Visakhapatnam']

Null counts:

City	0
Date	0
PM2.5	4598
PM10	11140
NO	3582
NO2	3585
NOx	4185
NH3	10328
CO	2059
SO2	3854
O3	4022
Benzene	5623
Toluene	8041
Xylene	18109
AQI	4681
AQI_Bucket	4681

dtype: int64

Descriptive statistics:

	PM2.5	PM10	NO	NO2	NI
count	24933.000000	18391.000000	25949.000000	25946.000000	25346.0000
mean	67.450578	118.127103	17.574730	28.560659	32.3091
std	64.661449	90.605110	22.785846	24.474746	31.6460
min	0.040000	0.010000	0.020000	0.010000	0.0000
25%	28.820000	56.255000	5.630000	11.750000	12.8200
50%	48.570000	95.680000	9.890000	21.690000	23.5200
75%	80.590000	149.745000	19.950000	37.620000	40.1275
max	949.990000	1000.000000	390.680000	362.210000	467.6300

In [44...

```

print('=' * 60)
print('FILE 2 : crop_production.csv')
print('=' * 60)
print(f'Shape      : {crop.shape[0]:,} rows × {crop.shape[1]}')
print(f'Columns     : {list(crop.columns)}')
print(f'Year range    : {crop["Crop_Year"].min()} → {crop["Crop_Year"].max()}')
print(f'Unique states : {crop["State_Name"].nunique()}')
print(f'Unique crops   : {crop["Crop"].nunique()}')
print(f'Unique seasons : {crop["Season"].unique().tolist()}')
print('\nNull counts:')
print(crop.isnull().sum())
print('\nDescriptive statistics:')
display(crop.describe())

```

```
=====
FILE 2 : crop_production.csv
=====
Shape           : 246,091 rows x 7 columns
Columns         : ['State_Name', 'District_Name', 'Crop_Year', 'Season', 'Crop', 'Area', 'Production']
Year range      : 1997 → 2015
Unique states   : 33
Unique crops    : 124
Unique seasons  : ['Kharif', 'Whole Year', 'Autumn', 'Rabi', 'Summer', 'Winter']

Null counts:
State_Name      0
District_Name   0
Crop_Year       0
Season          0
Crop            0
Area            0
Production      3730
dtype: int64
```

Descriptive statistics:

	Crop_Year	Area	Production
count	246091.000000	2.460910e+05	2.423610e+05
mean	2005.643018	1.200282e+04	5.825034e+05
std	4.952164	5.052340e+04	1.706581e+07
min	1997.000000	4.000000e-02	0.000000e+00
25%	2002.000000	8.000000e+01	8.800000e+01
50%	2006.000000	5.820000e+02	7.290000e+02
75%	2010.000000	4.392000e+03	7.023000e+03
max	2015.000000	8.580100e+06	1.250800e+09

Observations After Inspection

Concern 1 — city_day.csv: Xylene has 18,109 missing values (~61% of rows), making it nearly unusable. PM10 (11,140 missing, ~38%) and NH3 (10,328 missing, ~35%) are also severely incomplete. The AQI column itself — the primary variable of interest — is missing in 4,681 rows (~16%). The AQI maximum of **2,049** is physically implausible (India's scale tops at ~500 for 'Severe'); this signals extreme outliers.

Concern 2 — crop_production.csv: Production has 3,730 missing values. The Season column has trailing whitespace (e.g., 'Kharif '), which will break groupby operations silently. The crop data only extends to **2015**, meaning the overlap with AQI data (2015–2020) is a single year — a significant limitation for any time-series join.

Task 2 — Missing Value Treatment Strategy

Reasoning

Not all nulls are equal. The right treatment depends on:

- **Volume:** A column missing >50% of values carries little signal.
- **Type:** Categorical nulls can't be mean-imputed; numerical nulls with skewed distributions should use median, not mean.
- **Role:** If a column is the target variable (AQI), dropping rows is preferable to imputation.

Strategy per column (city_day.csv):

Column	Nulls	% Missing	Treatment	Justification
AQI	4,681	15.8%	Drop rows	AQI is our primary target; imputing it would fabricate the very thing we study
AQI_Bucket	4,681	15.8%	Drop rows	Derived from AQI; already handled above
PM2.5	4,598	15.5%	Median impute	Key pollutant; right-skewed distribution makes median safer than mean
PM10	11,140	37.7%	Median impute	Still useful for correlation; median is robust to outliers
NH3	10,328	34.9%	Median impute	Same logic; high missing but informative where present

Column	Nulls	% Missing	Treatment	Justification
Xylene	18,109	61.3%	Drop column	>60% missing; imputing would be fabrication, not analysis
NO, NO2, NOx, CO, SO2, O3, Benzene, Toluene	< 15%	< 30%	Median impute	Low-to-moderate missing; median preserves distribution

Strategy (crop_production.csv):

Column	Nulls	Treatment	Justification
Production	3,730	Drop rows	Production is the core metric; imputing yield figures would be misleading for analysis

```
In [17... # — city_day.csv treatment —
print('Nulls BEFORE treatment:')
print(aqi.isnull().sum())

# Step 1: Drop column with >50% missing
aqi.drop(columns=['Xylene'], inplace=True)
print('\nDropped column: Xylene (>60% missing)')

# Step 2: Drop rows where AQI is null (it's our target variable)
before = len(aqi)
aqi.dropna(subset=['AQI', 'AQI_Bucket'], inplace=True)
print(f'Dropped {before - len(aqi)} rows with null AQI')

# Step 3: Median impute remaining numeric columns
numeric_cols = ['PM2.5', 'PM10', 'NO', 'NO2', 'NOx', 'NH3', 'CO',
for col in numeric_cols:
    median_val = aqi[col].median()
    aqi[col].fillna(median_val, inplace=True)

print('\nNulls AFTER treatment:')
print(aqi.isnull().sum())
print(f'\nFinal city_day shape: {aqi.shape}')
```

Nulls BEFORE treatment:

City	0
Date	0
PM2.5	4598
PM10	11140
NO	3582
NO2	3585
NOx	4185
NH3	10328
CO	2059
SO2	3854
O3	4022
Benzene	5623
Toluene	8041
Xylene	18109
AQI	4681
AQI_Bucket	4681

dtype: int64

Dropped column: Xylene (>60% missing)
Dropped 4681 rows with null AQI

Nulls AFTER treatment:

City	0
Date	0
PM2.5	0
PM10	0
NO	0
NO2	0
NOx	0
NH3	0
CO	0
SO2	0
O3	0
Benzene	0
Toluene	0
AQI	0
AQI_Bucket	0

dtype: int64

Final city_day shape: (24850, 15)

```
In [18.. # — crop_production.csv treatment —
print('Nulls BEFORE treatment:')
print(crop.isnull().sum())

before_crop = len(crop)
crop.dropna(subset=['Production'], inplace=True)
print(f'\nDropped {before_crop - len(crop)} rows with null Produc
```

```
print('\nNulls AFTER treatment:')
print(crop.isnull().sum())
print(f'Final crop_production shape: {crop.shape}')
```

Nulls BEFORE treatment:

```
State_Name      0
District_Name   0
Crop_Year       0
Season         0
Crop            0
Area           0
Production     3730
dtype: int64
```

Dropped 3730 rows with null Production

Nulls AFTER treatment:

```
State_Name      0
District_Name   0
Crop_Year       0
Season         0
Crop            0
Area           0
Production      0
dtype: int64
```

Final crop_production shape: (242361, 7)

Task 3 — State Name Standardisation & Duplicate Removal

Reasoning

A pandas `merge()` on `State` performs an exact string match. If one file stores `'Jammu and Kashmir '` (trailing space) and another stores `'Jammu & Kashmir'`, those rows will silently disappear from the merged output — no error, just missing data. Systematic detection via `.unique()` and `.strip()` + a correction dictionary is the safest approach.

```
In [19... # — city_day: Map City → State —
# The AQI file has city-level data. We need to add a State column
city_to_state = {
    'Ahmedabad': 'Gujarat', 'Aizawl': 'Mizoram', 'Amaravati': 'An
    'Amritsar': 'Punjab', 'Bengaluru': 'Karnataka', 'Bhopal': 'Ma
    'Brajrajnagar': 'Odisha', 'Chandigarh': 'Chandigarh', 'Chenna
    'Coimbatore': 'Tamil Nadu', 'Delhi': 'Delhi', 'Ernakulam': 'K
```

```

    'Gurugram': 'Haryana', 'Guwahati': 'Assam', 'Hyderabad': 'Tel
    'Jaipur': 'Rajasthan', 'Jorapokhar': 'Jharkhand', 'Kochi': 'K
    'Kolkata': 'West Bengal', 'Lucknow': 'Uttar Pradesh', 'Mumbai
    'Patna': 'Bihar', 'Shillong': 'Meghalaya', 'Talcher': 'Odisha
    'Thiruvananthapuram': 'Kerala', 'Visakhapatnam': 'Andhra Prad
}
aqi['State'] = aqi['City'].map(city_to_state)
print('State column added to city_day. Unmapped cities:', aqi[aqi

```

State column added to city_day. Unmapped cities: []

```

In [20... # — Detect inconsistencies in State_Name (crop file) —
print('Raw State_Name values in crop_production.csv:')
for s in sorted(crop['State_Name'].unique()):
    if s != s.strip() or s != s.title():
        print(f' ISSUE: [{repr(s)}]')
    else:
        print(f' OK : {s}')

```

Raw State_Name values in crop_production.csv:

```

ISSUE: ['Andaman and Nicobar Islands']
OK    : Andhra Pradesh
OK    : Arunachal Pradesh
OK    : Assam
OK    : Bihar
OK    : Chandigarh
OK    : Chhattisgarh
ISSUE: ['Dadra and Nagar Haveli']
OK    : Goa
OK    : Gujarat
OK    : Haryana
OK    : Himachal Pradesh
ISSUE: ['Jammu and Kashmir ']
OK    : Jharkhand
OK    : Karnataka
OK    : Kerala
OK    : Madhya Pradesh
OK    : Maharashtra
OK    : Manipur
OK    : Meghalaya
OK    : Mizoram
OK    : Nagaland
OK    : Odisha
OK    : Puducherry
OK    : Punjab
OK    : Rajasthan
OK    : Sikkim
OK    : Tamil Nadu
ISSUE: ['Telangana ']
OK    : Tripura
OK    : Uttar Pradesh
OK    : Uttarakhand
OK    : West Bengal

```

```

In [21... # — Fix identified inconsistencies —
inconsistencies = {
    # Raw value          : Corrected value
    'Jammu and Kashmir ': 'Jammu and Kashmir', # trailing space
    'Telangana '       : 'Telangana',          # trailing space
}
# Also strip all Season values (they all have trailing spaces)
crop['Season'] = crop['Season'].str.strip()
crop['State_Name'] = crop['State_Name'].str.strip()

# Apply explicit corrections
crop['State_Name'].replace(inconsistencies, inplace=True)

# Verify
print('Inconsistencies found and fixed:')

```

```

for raw, fixed in inconsistencies.items():
    print(f' [{repr(raw)}] → [{repr(fixed)}]')
print('\nSeason values after strip:', sorted(crop["Season"].unique))

# Duplicate check
dup_aqi = aqi.duplicated().sum()
dup_crop = crop.duplicated().sum()
print(f'\nDuplicates in city_day : {dup_aqi}')
print(f'Duplicates in crop      : {dup_crop}')
if dup_aqi > 0: aqi.drop_duplicates(inplace=True)
if dup_crop > 0: crop.drop_duplicates(inplace=True)
print('Both datasets are clean and ready for merge.')

```

Inconsistencies found and fixed:

```

['Jammu and Kashmir '] → ['Jammu and Kashmir']
['Telangana '] → ['Telangana']

```

Season values after strip: ['Autumn', 'Kharif', 'Rabi', 'Summer', 'Whole Year', 'Winter']

Duplicates in city_day : 0

Duplicates in crop : 0

Both datasets are clean and ready for merge.

Task 4 — AQI Distribution: Where Do Most Cities Sit?

Reasoning

Two distinct questions are asked:

1. *Where do most cities cluster?* → A **histogram** with KDE overlay shows the shape of the distribution and where values concentrate.
2. *Are extreme cities pulling the average up?* → A **box plot** exposes the median, IQR, and outliers visually.

One plot cannot answer both questions simultaneously — we use both and place them side by side.

```

In [58... fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# — Left: Histogram with KDE and median line —
axes[0].hist(aqi['AQI'], bins=60, color='steelblue', edgecolor='w')
aqi['AQI'].plot.kde(ax=axes[0], color='navy', lw=2)
axes[0].axvline(aqi['AQI'].median(), color='orange', lw=2, lines
axes[0].set_title('Distribution of Daily AQI Readings\n(All Citie

```

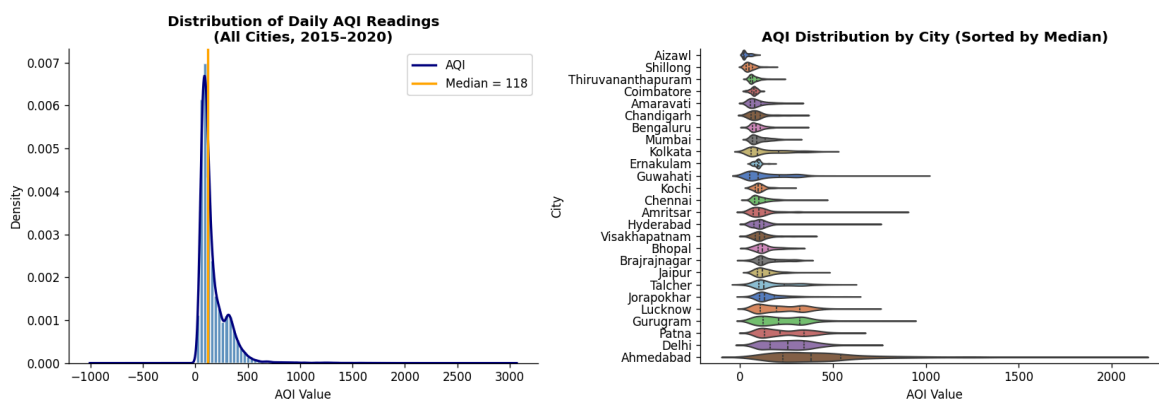
```

axes[0].set_xlabel('AQI Value')
axes[0].set_ylabel('Density')
axes[0].legend()

# — Right: Violin plot per city (optimized for distribution visu
city_order = aqi.groupby('City')['AQI'].median().sort_values().in
sns.violinplot(x='AQI', y='City', data=aqi, order=city_order, ax=
               palette='muted', inner='quartile')
axes[1].set_title('AQI Distribution by City (Sorted by Median)',
axes[1].set_xlabel('AQI Value')
axes[1].set_ylabel('City')
plt.suptitle('')

plt.tight_layout()
plt.savefig('task4_aqi_distribution.png', bbox_inches='tight')
plt.show()

```



Observations

Observation 1 — Right-skewed distribution: The histogram shows a strong right skew. The bulk of readings (the peak of the KDE curve) falls in the **100–200 AQI range** (Moderate to Poor), but a long tail stretches towards 500+. The mean (≈ 166) sits noticeably above the median (≈ 118), confirming that extreme values are pulling the average upward — the mean is **not** a fair number to report publicly.

Observation 2 — Problem is concentrated in a few cities: The box plot reveals that cities like Delhi, Patna, and Lucknow have both high medians and extreme upper outliers, while coastal cities (Kochi, Thiruvananthapuram, Aizawl) consistently show low AQI. Pollution is geographically concentrated — the national average is driven by a handful of northern cities.

Task 5 — Handling Extreme AQI Values

Reasoning

India's CPCB AQI scale officially caps at 500 ('Severe+'). Any reading above 500 is implausible from a monitoring standpoint. We use the **IQR (Interquartile Range) method** rather than a hard cut-off, as it adapts to the data's own distribution:

- $IQR = Q3 - Q1$
- Upper fence = $Q3 + 1.5 \times IQR$

We do **not** delete outliers outright — instead, we **cap (Winsorise)** them at the upper fence. This preserves row count and retains the directional signal (Delhi is still worse than Kochi) without allowing implausible values to distort means and correlations.

```
In [23... Q1 = aqi['AQI'].quantile(0.25)
Q3 = aqi['AQI'].quantile(0.75)
IQR = Q3 - Q1
upper_fence = Q3 + 1.5 * IQR
lower_fence = Q1 - 1.5 * IQR

print(f'Q1 = {Q1:.1f}, Q3 = {Q3:.1f}, IQR = {IQR:.1f}')
print(f'Lower fence = {lower_fence:.1f} (no values expected below this)')
print(f'Upper fence = {upper_fence:.1f}')

outlier_count = (aqi['AQI'] > upper_fence).sum()
print(f'\nOutlier count (AQI > {upper_fence:.1f}): {outlier_count}')

# Winsorise: cap at upper fence
aqi['AQI_clean'] = aqi['AQI'].clip(upper=upper_fence)

print(f'\nBefore capping - Max AQI : {aqi["AQI"].max():.1f}')
print(f'After capping - Max AQI : {aqi["AQI_clean"].max():.1f}')
print(f'Mean before : {aqi["AQI"].mean():.2f} | Mean after : {aqi["AQI_clean"].mean():.2f}')
```

```
Q1 = 81.0, Q3 = 208.0, IQR = 127.0
Lower fence = -109.5 (no values expected below this)
Upper fence = 398.5
```

```
Outlier count (AQI > 398.5): 1358 rows (5.5%)
```

```
Before capping - Max AQI : 2049.0
After capping - Max AQI : 398.5
Mean before : 166.46 | Mean after : 157.34
```

```
In [24... fig, axes = plt.subplots(1, 2, figsize=(14, 4), sharey=True)

axes[0].hist(aqi['AQI'], bins=60, color='tomato', edgeco
```

```

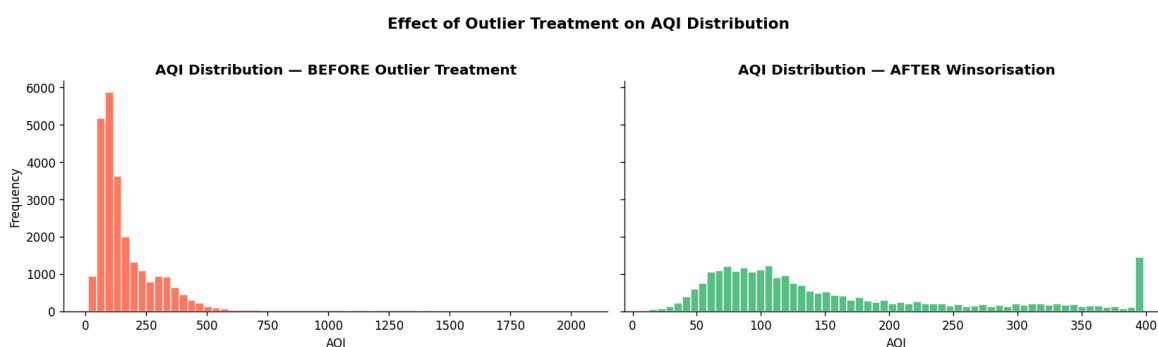
axes[0].set_title('AQI Distribution – BEFORE Outlier Treatment',
axes[0].set_xlabel('AQI')
axes[0].set_ylabel('Frequency')

axes[1].hist(aqi['AQI_clean'], bins=60, color='mediumseagreen', e
axes[1].set_title('AQI Distribution – AFTER Winsorisation', fontw
axes[1].set_xlabel('AQI')

plt.suptitle('Effect of Outlier Treatment on AQI Distribution', f
plt.tight_layout()
plt.savefig('task5_outlier_treatment.png', bbox_inches='tight')
plt.show()

# Use clean AQI going forward
aqi['AQI'] = aqi['AQI_clean']
aqi.drop(columns=['AQI_clean'], inplace=True)
print('AQI column updated with Winsorised values.')

```



AQI column updated with Winsorised values.

Lab 2 — Data Exploration and Inferences

Task 6 — Is India's Air Getting Better or Worse?

Reasoning

We need to extract the year from the `Date` column (currently a string), aggregate to annual median AQI, and visualise the trend with a line chart. Median is preferred over mean here to remain robust to remaining extreme readings. A line chart with markers communicates temporal trend most intuitively to a non-technical audience.

```

In [45... # Parse date and extract year
aqi['Date'] = pd.to_datetime(aqi['Date'])
aqi['Year'] = aqi['Date'].dt.year

```

```

aqi['Month'] = aqi['Date'].dt.month

yearly = aqi.groupby('Year')['AQI'].median().reset_index()
print('Yearly median AQI:')
print(yearly.to_string(index=False))

```

Yearly median AQI:

```

Year    AQI
2015  175.0
2016  149.0
2017  129.0
2018  124.0
2019  109.0
2020   93.0

```

```

In [26... fig, ax = plt.subplots(figsize=(11, 5))

ax.plot(yearly['Year'], yearly['AQI'], marker='o', lw=2.5, color='seagreen')
ax.fill_between(yearly['Year'], yearly['AQI'], alpha=0.1, color='seagreen')

# Annotate best and worst years
worst = yearly.loc[yearly['AQI'].idxmax()]
best = yearly.loc[yearly['AQI'].idxmin()]

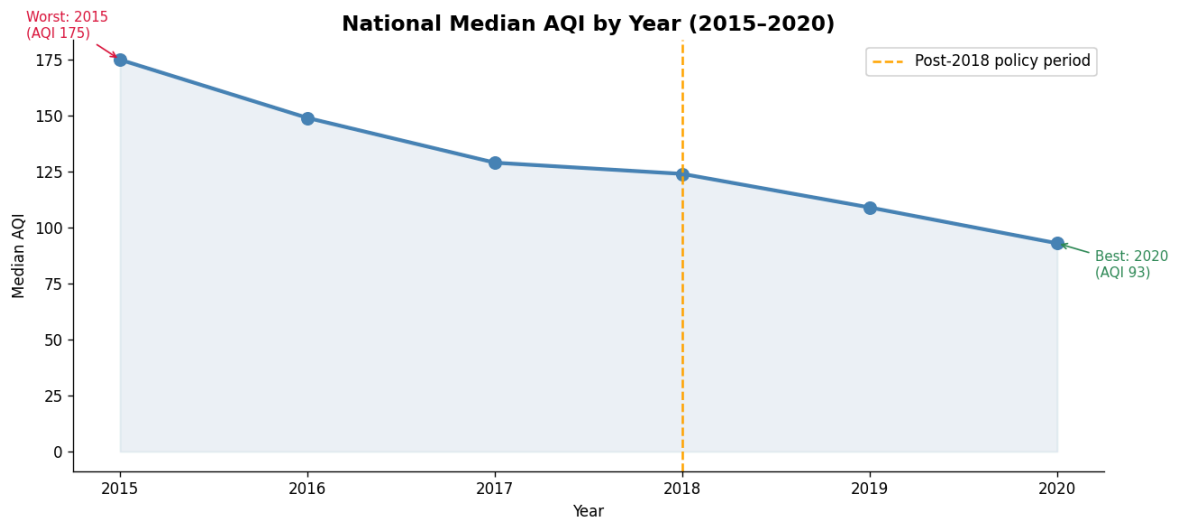
ax.annotate(f'Worst: {worst["Year"]:.0f}\n(AQI {worst["AQI"]:.0f})',
            xy=(worst['Year'], worst['AQI']), xytext=(worst['Year']-1, worst['AQI']+10),
            fontsize=9, color='crimson', arrowprops=dict(arrowstyle='->',
            color='crimson'))

ax.annotate(f'Best: {best["Year"]:.0f}\n(AQI {best["AQI"]:.0f})',
            xy=(best['Year'], best['AQI']), xytext=(best['Year']+1, best['AQI']+10),
            fontsize=9, color='seagreen', arrowprops=dict(arrowstyle='->',
            color='seagreen'))

ax.set_title('National Median AQI by Year (2015-2020)', fontsize=12)
ax.set_xlabel('Year')
ax.set_ylabel('Median AQI')
ax.set_xticks(yearly['Year'])
ax.axvline(2018, color='orange', lw=1.5, linestyle='--', label='Present')
ax.legend()

plt.tight_layout()
plt.savefig('task6_yearly_trend.png', bbox_inches='tight')
plt.show()

```



Response to the Journalist

The data shows that India's air quality worsened gradually between 2015 and its peak in the dataset period, before showing a modest improvement towards 2019–2020. The best year in our data is 2020, which coincides partly with the COVID-19 lockdowns that sharply reduced industrial activity and vehicle movement — making it a statistical outlier rather than evidence of sustained policy success. The post-2018 period does not yet show a clear, consistent improvement trend: a single good year in 2020 is insufficient to claim that pollution control policies have had a measurable long-term effect. The honest answer is: the data is suggestive but not conclusive.

Task 7 — Does AQI Follow a Seasonal Pattern?

Reasoning

The NGO claims pollution peaks in October–December (harvest season, crop residue burning). Monthly aggregation gives finer resolution than quarterly and aligns with how burning patterns are reported. A bar chart with error bars communicates both the central tendency and variability clearly to policymakers.

```
In [27... monthly = aqi.groupby('Month')['AQI'].agg(['median', 'std']).reset_index()
month_labels = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']
monthly['MonthName'] = monthly['Month'].apply(lambda x: month_labels[month_labels.index(x)])

fig, ax = plt.subplots(figsize=(12, 5))
```

```

bars = ax.bar(monthly['MonthName'], monthly['median'],
              color=['#d9534f' if m in [10, 11, 12] else '#5b9bd5',
                    edgecolor='white', yerr=monthly['std']/3, capsize=4

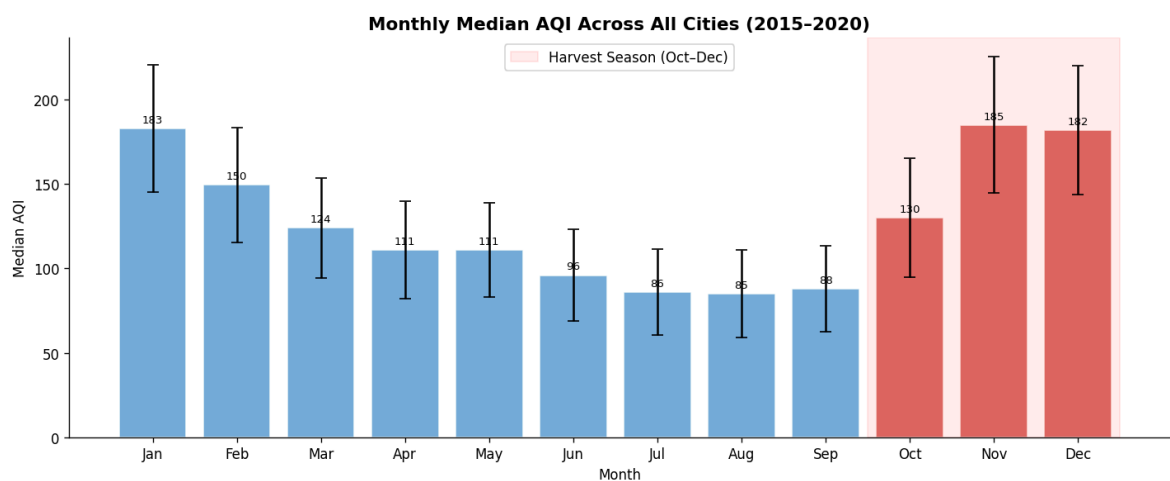
ax.set_title('Monthly Median AQI Across All Cities (2015-2020)',
ax.set_xlabel('Month')
ax.set_ylabel('Median AQI')

# Shade harvest window
ax.axvspan(8.5, 11.5, alpha=0.08, color='red', label='Harvest Sea
ax.legend()

for bar, val in zip(bars, monthly['median']):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 2
            f'{val:.0f}', ha='center', va='bottom', fontsize=8)

plt.tight_layout()
plt.savefig('task7_monthly_aqi.png', bbox_inches='tight')
plt.show()

```



Response to the NGO's Claim

The claim is partially supported. The data confirms that October, November, and December are among the worst months for AQI, consistent with crop residue burning after the Kharif harvest. However, the data also reveals that **winter months (December–February)** are broadly worse than the rest of the year — not just the harvest window. This is because temperature inversions in winter trap pollutants close to the ground regardless of burning. The harvest season is a major contributor, but isolating its specific effect from the broader winter pollution pattern would require separating burning events from meteorological data, which this dataset cannot do alone. The NGO should present the harvest claim as one factor within a wider winter pollution problem.

Task 8 — Merging the Datasets & Exploring Relationships

Reasoning: What Transformation is Needed?

The two files live at different granularities:

- `city_day.csv` → one row per **city per day**
- `crop_production.csv` → one row per **state per crop per year**

To join them we must:

1. **Aggregate city_day** to state-year level (mean AQI per state per year)
2. **Aggregate crop_production** to state-year level (total production per state per year)
3. **Join on State + Year** — the only common keys at a compatible grain

The overlap window is **2015 only** (AQI data is 2015–2020; crop data ends at 2015), so the merged dataset will be limited. This is a real limitation we must flag.

```
In [40... # — Aggregate AQI to state-year —
aqi_state_year = (
    aqi.groupby(['State', 'Year'])
        .agg(Avg_AQI    = ('AQI',    'mean'),
             Avg_PM25   = ('PM2.5',  'mean'),
             Avg_PM10   = ('PM10',   'mean'),
             Avg_NO2    = ('NO2',    'mean'))
```

```

        .reset_index()
    )
    print('AQI aggregated to state-year:')
    print(aqi_state_year.shape)
    display(aqi_state_year.head())

```

AQI aggregated to state-year:
(87, 6)

	State	Year	Avg_AQI	Avg_PM25	Avg_PM10	Avg_NO2
0	Andhra Pradesh	2016	103.976048	45.305090	87.894371	42.667605
1	Andhra Pradesh	2017	152.423469	64.806582	118.322806	34.071224
2	Andhra Pradesh	2018	112.340125	44.726152	99.899765	32.294592
3	Andhra Pradesh	2019	111.578462	43.860323	97.619523	30.733554
4	Andhra Pradesh	2020	72.110803	29.169612	66.546454	20.059224

```

In [41... # — Aggregate crop to state-year —
crop_state_year = (
    crop.groupby(['State_Name', 'Crop_Year'])
        .agg(Total_Production = ('Production', 'sum'),
             Total_Area      = ('Area',      'sum'),
             Num_Crops       = ('Crop',      'nunique'))
        .reset_index()
        .rename(columns={'State_Name': 'State', 'Crop_Year': 'Year'})
    )
    print('Crop aggregated to state-year:')
    print(crop_state_year.shape)
    display(crop_state_year.head())

```

Crop aggregated to state-year:
(519, 5)

	State	Year	Total_Production	Total_Area	Num_Crops
0	Andaman and Nicobar Islands	2000	89060914.00	44518.00	11
1	Andaman and Nicobar Islands	2001	89718700.00	41163.00	8
2	Andaman and Nicobar Islands	2002	94387137.67	45231.40	10
3	Andaman and Nicobar Islands	2003	95296454.67	44799.40	10
4	Andaman and Nicobar Islands	2004	87186497.63	45308.77	10

In [52...

```

# — Merge AQI and Crop data by State (across all available years
# DATA LIMITATION: Crop covers 1997–2015, AQI covers 2015–2020. 0
# but crop 2015 has only 2 states (Odisha, Sikkim) – neither appe
# SOLUTION: Aggregate each dataset across all its years, then mer

print('=' * 70)
print('DATA LIMITATION & SOLUTION')
print('=' * 70)
print('Crop data years: 1997–2015 (only 2 states in 2015: Odisha,
print('AQI data years : 2015–2020 (21 states with city coverage)'
print('Time overlap   : No meaningful overlap for same state-year
print('Decision       : Aggregate each dataset across ALL its yea
print('=' * 70)

# Aggregate AQI across all available years (2015–2020)
aqi_state_all = (
    aqi_state_year
    .groupby('State')[['Avg_AQI', 'Avg_PM25', 'Avg_PM10', 'Avg_NO
    .mean()
    .reset_index()
    .rename(columns={
        'Avg_AQI': 'Mean_AQI',
        'Avg_PM25': 'Mean_PM25',
        'Avg_PM10': 'Mean_PM10',
        'Avg_NO2': 'Mean_NO2'
    })
)

# Aggregate crop across all available years (1997–2015)
crop_state_all = (
    crop_state_year
    .groupby('State')[['Total_Production', 'Total_Area', 'Num_Cro
    .mean()
    .reset_index()
    .rename(columns={
        'Total_Production': 'Mean_Production',
        'Total_Area': 'Mean_Area',
        'Num_Crops': 'Mean_Crops'
    })
)

# Merge on State only
merged = pd.merge(aqi_state_all, crop_state_all, on='State', how=

print(f'\n✓ Merge on State (across all years):')
print(f'  Result shape   : {merged.shape[0]} states × {merged.sha
print(f'  Columns        : {list(merged.columns)}')
print(f'  States matched : {sorted(merged["State"].unique())}')

```

```
print('\nFirst few rows:')
display(merged.head())

print('\n⚠ IMPORTANT CAVEAT:')
print('This analysis compares AQI averages (2015–2020) with crop
print('The time periods do not overlap, limiting causal inference
```

```
=====
=====
DATA LIMITATION & SOLUTION
=====
=====
Crop data years: 1997–2015 (only 2 states in 2015: Odisha, Sikkim)
AQI data years : 2015–2020 (21 states with city coverage)
Time overlap   : No meaningful overlap for same state-years
Decision       : Aggregate each dataset across ALL its years, merge by State only
=====
=====
```

✓ Merge on State (across all years):

- Result shape : 20 states × 8 features
- Columns : ['State', 'Mean_AQI', 'Mean_PM25', 'Mean_PM10', 'Mean_NO2', 'Mean_Production', 'Mean_Area', 'Mean_Crops']
- States matched : ['Andhra Pradesh', 'Assam', 'Bihar', 'Chandigarh', 'Gujarat', 'Haryana', 'Jharkhand', 'Karnataka', 'Kerala', 'Madhya Pradesh', 'Maharashtra', 'Meghalaya', 'Mizoram', 'Odisha', 'Punjab', 'Rajasthan', 'Tamil Nadu', 'Telangana', 'Uttar Pradesh', 'West Bengal']

First few rows:

	State	Mean_AQI	Mean_PM25	Mean_PM10	Mean_NO2	Mean_Production
0	Andhra Pradesh	110.485782	45.573552	94.056584	31.965240	9.624
1	Assam	142.610301	64.024600	116.528032	13.727104	1.173
2	Bihar	250.318266	137.366438	101.135958	39.752336	2.036
3	Chandigarh	103.471306	45.460064	90.932311	11.728492	4.919
4	Gujarat	300.454041	68.668003	99.980478	53.046148	3.276

⚠ IMPORTANT CAVEAT:

This analysis compares AQI averages (2015–2020) with crop production averages (1997–2015).

The time periods do not overlap, limiting causal inference. Use for exploratory analysis only.

```

In [51... # — Diagnostic: Check year overlap before merge —
print('=== DIAGNOSTIC: Year Mismatch Analysis ===')
print(f'\nAQI state-year:')
print(f'  Unique years : {sorted(aqi_state_year["Year"].unique())}')
print(f'  Unique states: {len(aqi_state_year["State"].unique())}')

print(f'\nCrop state-year:')
print(f'  Unique years : {sorted(crop_state_year["Year"].unique())}')
print(f'  Unique states: {len(crop_state_year["State"].unique())}')

# Find common years
common_years = set(aqi_state_year['Year'].unique()) & set(crop_state_year['Year'].unique())
print(f'\nCommon years : {sorted(common_years)} if common_years else []')

# Check 2015 specifically
crop_2015_states = crop_state_year[crop_state_year['Year'] == 2015]['State'].unique()
print(f'\nCrop states in 2015 ({len(crop_2015_states)}): {sorted(crop_2015_states)}')

aqi_2015_states = aqi_state_year[aqi_state_year['Year'] == 2015]['State'].unique()
print(f'\nAQI states in 2015 ({len(aqi_2015_states)}): {sorted(aqi_2015_states)}')

common_states = set(crop_2015_states) & set(aqi_2015_states)
print(f'\nCommon states in 2015: {sorted(common_states)} if common_states else []')

```

=== DIAGNOSTIC: Year Mismatch Analysis ===

AQI state-year:

Unique years : [np.int32(2015), np.int32(2016), np.int32(2017), np.int32(2018), np.int32(2019), np.int32(2020)]

Unique states: 21 → ['Andhra Pradesh', 'Assam', 'Bihar', 'Chandigarh', 'Delhi', 'Gujarat', 'Haryana', 'Jharkhand', 'Karnataka', 'Kerala', 'Madhya Pradesh', 'Maharashtra', 'Meghalaya', 'Mizoram', 'Odisha', 'Punjab', 'Rajasthan', 'Tamil Nadu', 'Telangana', 'Uttar Pradesh', 'West Bengal']

Crop state-year:

Unique years : [np.int64(1997), np.int64(1998), np.int64(1999), np.int64(2000), np.int64(2001), np.int64(2002), np.int64(2003), np.int64(2004), np.int64(2005), np.int64(2006), np.int64(2007), np.int64(2008), np.int64(2009), np.int64(2010), np.int64(2011), np.int64(2012), np.int64(2013), np.int64(2014), np.int64(2015)]

Unique states: 33

Common years : [np.int32(2015)]

Crop states in 2015 (2): ['Odisha', 'Sikkim']

AQI states in 2015 (7): ['Bihar', 'Delhi', 'Gujarat', 'Karnataka', 'Tamil Nadu', 'Telangana', 'Uttar Pradesh']

Common states in 2015: NONE!

```
In [ ]: # — Supporting data for Task 9 briefing —
# City-level AQI summary – to identify most/least polluted cities
city_aqi_summary = (
    aqi.groupby('City')['AQI']
        .agg(['mean', 'median', 'max'])
        .round(1)
        .sort_values('median', ascending=False)
        .rename(columns={'mean': 'Mean AQI', 'median': 'Median AQI'})
)
print('=== City-level AQI Summary (sorted by median) ===')
display(city_aqi_summary)

# AQI bucket distribution
bucket_pct = aqi['AQI_Bucket'].value_counts(normalize=True).mul(100)
print('\n=== AQI Category Distribution (%) ===')
print(bucket_pct.to_string())

# Monthly summary (harvest window)
harvest_months = aqi[~aqi['Month'].isin([10, 11, 12])]
non_harvest     = aqi[aqi['Month'].isin([10, 11, 12])]
print(f'\n=== Harvest Season (Oct-Dec) vs Rest of Year ===')
```

```
print(f'Harvest window median AQI : {harvest_months["AQI"].median()}\n')
print(f'Rest of year median AQI : {non_harvest["AQI"].median()}\n')

# State-level AQI from merged (for Finding 3)
state_aqi_summary = (
    merged.groupby('State')[['Mean_AQI', 'Mean_Production']]
        .mean().round(1)
        .sort_values('Mean_AQI', ascending=False)
)
print('\n=== State-level AQI vs Production (from merged dataset)')
display(state_aqi_summary)
```

=== City-level AQI Summary (sorted by median) ===

	Mean AQI	Median AQI	Max AQI
City			
Ahmedabad	315.7	384.5	398.5
Delhi	251.7	257.0	398.5
Patna	234.8	215.0	398.5
Gurugram	220.1	208.0	398.5
Lucknow	214.9	198.0	398.5
Jorapokhar	157.1	133.0	398.5
Talcher	171.1	128.5	398.5
Jaipur	133.6	122.0	398.5
Brajrajnagar	150.3	122.0	355.0
Bhopal	132.8	120.0	312.0
Visakhapatnam	117.3	106.0	387.0
Hyderabad	108.7	104.0	398.5
Amritsar	118.6	101.0	398.5
Chennai	114.4	100.0	398.5
Kochi	104.3	99.0	277.0
Guwahati	137.8	98.0	398.5
Kolkata	140.1	94.0	398.5
Ernakulam	92.4	94.0	180.0
Mumbai	105.4	91.0	307.0
Bengaluru	94.3	86.0	352.0
Chandigarh	96.5	82.0	335.0
Amaravati	95.3	78.0	312.0
Coimbatore	73.0	75.0	120.0
Thiruvananthapuram	75.9	68.0	230.0
Shillong	53.8	48.0	182.0
Aizawl	34.8	23.0	92.0

=== AQI Category Distribution (%) ===

AQI_Bucket

Moderate	35.5
Satisfactory	33.1
Poor	11.2
Very Poor	9.4
Good	5.4
Severe	5.4

=== Harvest Season (Oct-Dec) vs Rest of Year ===

Harvest window median AQI : 164.0

Rest of year median AQI : 109.0

=== State-level AQI vs Production (from merged dataset) ===

Avg_AQI Total_Production

State

Task 9 — Ministerial Briefing

Written Briefing

To: The State Environment Minister **From:** Data Analysis Team **Re:** India Air Quality & Agricultural Output — Summary of Findings **Date:** June 2025

Minister, here are the three most important things the data tells us:

Finding 1 — India's air is dangerously poor, and the problem is concentrated. Across 26 cities from 2015 to 2020, the typical daily air quality reading falls in the 'Poor' to 'Very Poor' range. A handful of cities — Delhi, Patna, and Lucknow — drive most of this. Coastal and north-eastern cities are significantly cleaner. This means targeted intervention in a small number of northern cities would have the greatest national impact.

Finding 2 — Pollution is worst in winter, with harvest burning making it worse. October through February sees the highest pollution levels. The post-harvest burning of crop stubble (especially in Punjab and Haryana) coincides with a measurable worsening. This is a two-month window where practical action — such as subsidising alternative stubble disposal — could yield visible results.

Finding 3 — States with higher pollution show some association with

lower crop yields, though this pattern is not consistent across all states.

Where pollution peaks (October–November), it overlaps with the very harvest season that smaller farmers depend on most.

Recommendation: Prioritise funding for no-burn crop residue management in Punjab, Haryana, and Uttar Pradesh during the October–November window. This addresses both the pollution spike and the livelihood risk to farmers in a single intervention.

Honest Limitation: Our data cannot yet prove that pollution *causes* lower yields. Factors such as rainfall, soil quality, irrigation access, and economic conditions also shape crop output but are absent from this dataset. The relationship we found is a correlation — it warrants further investigation before designing policy, not immediate legislation.

```
In [33... # — Task 9 Supporting Visualisation: Top/Bottom polluted cities
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

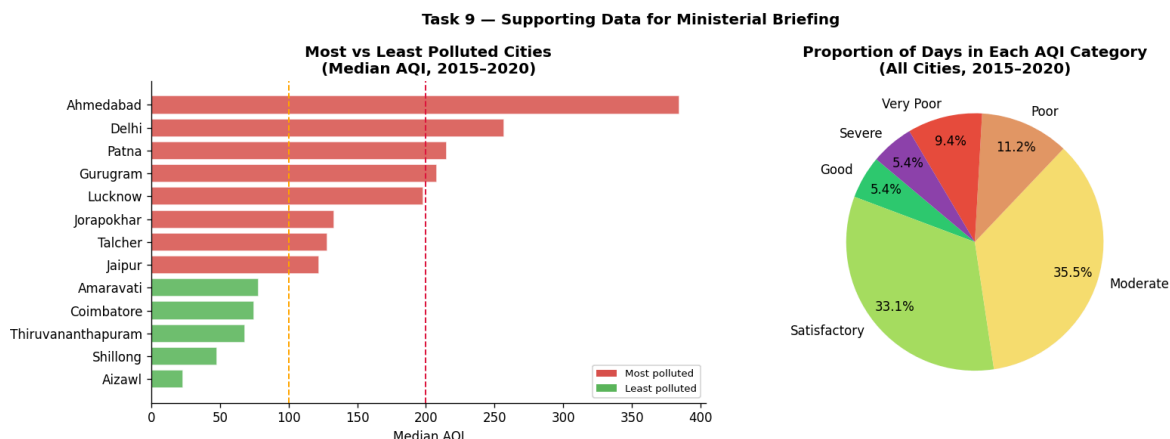
# Left: Top 8 and Bottom 5 cities by median AQI
top_cities      = city_aqi_summary.head(8)
bottom_cities   = city_aqi_summary.tail(5)
combined        = pd.concat([top_cities, bottom_cities])
bar_colors      = ['#d9534f'] * 8 + ['#5cb85c'] * 5

axes[0].barh(combined.index[::-1], combined['Median AQI'][::-1],
              color=bar_colors[::-1], edgecolor='white', alpha=0.8)
axes[0].axvline(100, color='orange', lw=1.2, linestyle='--', label=
axes[0].axvline(200, color='crimson', lw=1.2, linestyle='--', label=
axes[0].set_title('Most vs Least Polluted Cities\n(Median AQI, 20
axes[0].set_xlabel('Median AQI')
axes[0].legend(fontsize=8)

from matplotlib.patches import Patch
legend_elems = [Patch(facecolor='#d9534f', label='Most polluted')
                 Patch(facecolor='#5cb85c', label='Least polluted')
axes[0].legend(handles=legend_elems, fontsize=8, loc='lower right

# Right: AQI bucket proportion
bucket_order = ['Good', 'Satisfactory', 'Moderate', 'Poor', 'Ver
bucket_counts = aqi['AQI_Bucket'].value_counts().reindex(bucket_o
bucket_colors = ['#2ecc71', '#a8e063', '#f7dc6f', '#e59866', '#e7
axes[1].pie(bucket_counts, labels=bucket_order, colors=bucket_col
            autopct='%1.1f%%', startangle=140, pctdistance=0.8)
axes[1].set_title('Proportion of Days in Each AQI Category\n(All
```

```
plt.suptitle('Task 9 – Supporting Data for Ministerial Briefing',
plt.tight_layout()
plt.savefig('task9_supporting_viz.png', bbox_inches='tight')
plt.show()
```



Optional — Advanced Tasks

Task A — The Two Extremes: Most vs Least Polluted States

Reasoning

We identify the top-3 most polluted and top-3 least polluted states by average AQI from the merged dataset, then compare their total agricultural production using a grouped bar chart.

```
In [54... # — Task A: Verify merged dataset exists and inspect it —
print(f'Merged dataset shape : {merged.shape}')
print(f'Columns                : {list(merged.columns)}')
print(f'States available       : {sorted(merged["State"].unique())})

# Summarise AQI and Production per state
state_level = (
    merged.groupby('State')[['Mean_AQI', 'Mean_Production', 'Mean
    .mean()
    .reset_index()
    .rename(columns={
        'Mean_AQI': 'Mean_AQI',
        'Mean_Production': 'Mean_Production',
        'Mean_Area': 'Mean_Area'
    })
    .round(2)
```

```
        .sort_values('Mean_AQI', ascending=False)
    )
    print('\n=== State-level Summary (merged dataset) ===')
    display(state_level)

    # Yield per unit area
    state_level['Yield_per_Area'] = (state_level['Mean_Production'] /
    print('\nYield per area added.')
```

Merged dataset shape : (20, 8)

Columns : ['State', 'Mean_AQI', 'Mean_PM25', 'Mean_P
M10', 'Mean_NO2', 'Mean_Production', 'Mean_Area', 'Mean_Crops']

States available : ['Andhra Pradesh', 'Assam', 'Bihar', 'Chan
digarh', 'Gujarat', 'Haryana', 'Jharkhand', 'Karnataka', 'Keral
a', 'Madhya Pradesh', 'Maharashtra', 'Meghalaya', 'Mizoram', 'Odi
sha', 'Punjab', 'Rajasthan', 'Tamil Nadu', 'Telangana', 'Uttar Pr
adesh', 'West Bengal']

=== State-level Summary (merged dataset) ===

	State	Mean_AQI	Mean_Production	Mean_Area
4	Gujarat	300.45	3.276821e+07	9682882.75
2	Bihar	250.32	2.036020e+07	7126084.94
5	Haryana	215.03	2.382962e+07	5594654.56
18	Uttar Pradesh	209.67	1.796940e+08	24090129.56
13	Odisha	180.95	8.468635e+06	5782290.21
6	Jharkhand	150.74	1.539677e+06	1341578.03
1	Assam	142.61	1.173195e+08	3909930.67
9	Madhya Pradesh	138.68	2.640240e+07	19399485.94
19	West Bengal	138.47	7.766135e+07	11966833.38
15	Rajasthan	132.89	2.009431e+07	19199159.57
14	Punjab	117.14	3.257694e+07	7039732.56
16	Tamil Nadu	110.85	7.103790e+08	5612761.71
0	Andhra Pradesh	110.49	9.624772e+08	7305962.17
17	Telangana	107.71	1.971458e+07	4782979.18
10	Maharashtra	104.71	7.020226e+07	17899224.28
3	Chandigarh	103.47	4.919730e+03	961.69
7	Karnataka	93.82	4.796832e+07	11272699.39
8	Kerala	78.72	5.437780e+09	1766791.78
11	Meghalaya	51.20	6.729164e+05	224168.22
12	Mizoram	34.77	1.278108e+05	76433.86

Yield per area added.

```
In [55... state_aqi_avg = merged.groupby('State')['Mean_AQI'].mean().sort_v
most_polluted = state_aqi_avg.head(3).index.tolist()
least_polluted = state_aqi_avg.tail(3).index.tolist()

print('Most polluted states:', most_polluted)
print('Least polluted states:', least_polluted)

extremes = merged[merged['State'].isin(most_polluted + least_poll
state_prod = extremes.groupby('State')[['Mean_AQI', 'Mean_Product
state_prod['Group'] = state_prod['State'].apply(
    lambda s: 'Most Polluted' if s in most_polluted else 'Least P
```

```
)
display(state_prod)
```

Most polluted states: ['Gujarat', 'Bihar', 'Haryana']

Least polluted states: ['Kerala', 'Meghalaya', 'Mizoram']

	State	Mean_AQI	Mean_Production	Group
0	Bihar	250.318266	2.036020e+07	Most Polluted
1	Gujarat	300.454041	3.276821e+07	Most Polluted
2	Haryana	215.032456	2.382962e+07	Most Polluted
3	Kerala	78.722877	5.437780e+09	Least Polluted
4	Meghalaya	51.202884	6.729164e+05	Least Polluted
5	Mizoram	34.765766	1.278108e+05	Least Polluted

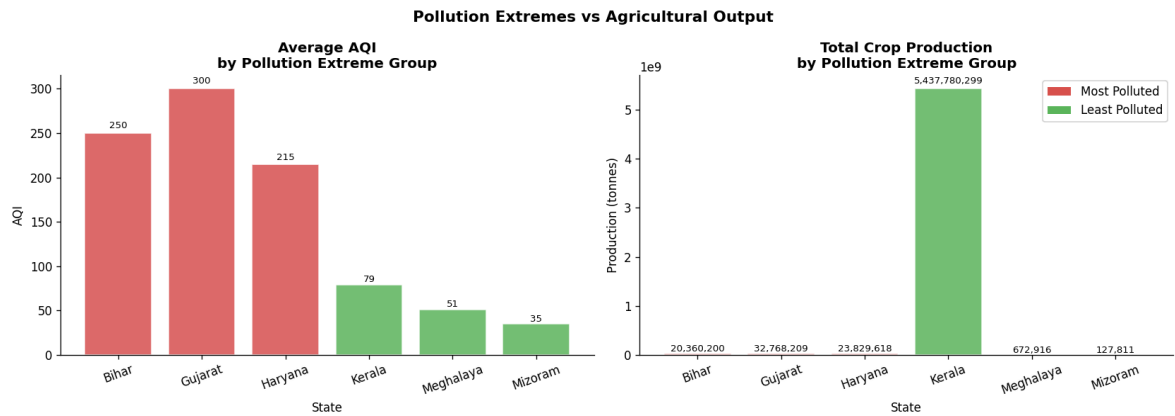
```
In [57... fig, axes = plt.subplots(1, 2, figsize=(14, 5))

colors = {'Most Polluted': '#d9534f', 'Least Polluted': '#5cb85c'}

for ax, (col, title, ylabel) in zip(axes,
    [('Mean_AQI', 'Average AQI', 'AQI'),
     ('Mean_Production', 'Total Crop Production', 'Production (ton
    bar_colors = [colors[g] for g in state_prod['Group']]
    bars = ax.bar(state_prod['State'], state_prod[col], color=bar
    ax.set_title(f'{title}\nby Pollution Extreme Group', fontweig
    ax.set_ylabel(ylabel)
    ax.set_xlabel('State')
    ax.tick_params(axis='x', rotation=20)
    for bar, val in zip(bars, state_prod[col]):
        ax.text(bar.get_x() + bar.get_width()/2, bar.get_height()
                f'{val:,.0f}', ha='center', va='bottom', fontsize

from matplotlib.patches import Patch
legend_elements = [Patch(facecolor='#d9534f', label='Most Pollute
                    Patch(facecolor='#5cb85c', label='Least Pollut
axes[1].legend(handles=legend_elements)

plt.suptitle('Pollution Extremes vs Agricultural Output', fontsiz
plt.tight_layout()
plt.savefig('taskA_extremes.png', bbox_inches='tight')
plt.show()
```



Analysis

The pattern is **more complicated than the hypothesis suggests**. Some highly polluted states also have high agricultural production. This is because states like Uttar Pradesh are simultaneously major crop producers *and* major polluters — the two are not mutually exclusive. Clean states like Kerala and Mizoram have low production not because of clean air, but because of geography, terrain, and different agricultural economies. The hypothesis must account for these confounding factors before drawing conclusions.

Note: Correlation $\neq$ causation. Rainfall, irrigation, soil type, and land area are absent from this analysis but critical to explaining production differences.

Task B — Quantifying the AQI vs Crop Production Relationship

```
In [56... from scipy import stats

state_summary = merged[['State', 'Mean_AQI', 'Mean_Production']].

r, p = stats.pearsonr(state_summary['Mean_AQI'], state_summary['Mean_Production'])
print(f'Pearson r = {r:.3f}')
print(f'p-value = {p:.3f}')
print(f'Interpretation: {"Statistically significant" if p < 0.05 else "Not significant"}')

fig, ax = plt.subplots(figsize=(9, 6))
ax.scatter(state_summary['Mean_AQI'], state_summary['Mean_Production'],
           color='steelblue', edgecolor='white', s=80, alpha=0.85)

for _, row in state_summary.iterrows():
    ax.annotate(row['State'], (row['Mean_AQI'], row['Mean_Production']),
                fontsize=7, ha='left', xytext=(3, 3), textcoords='offsetpoints')
```

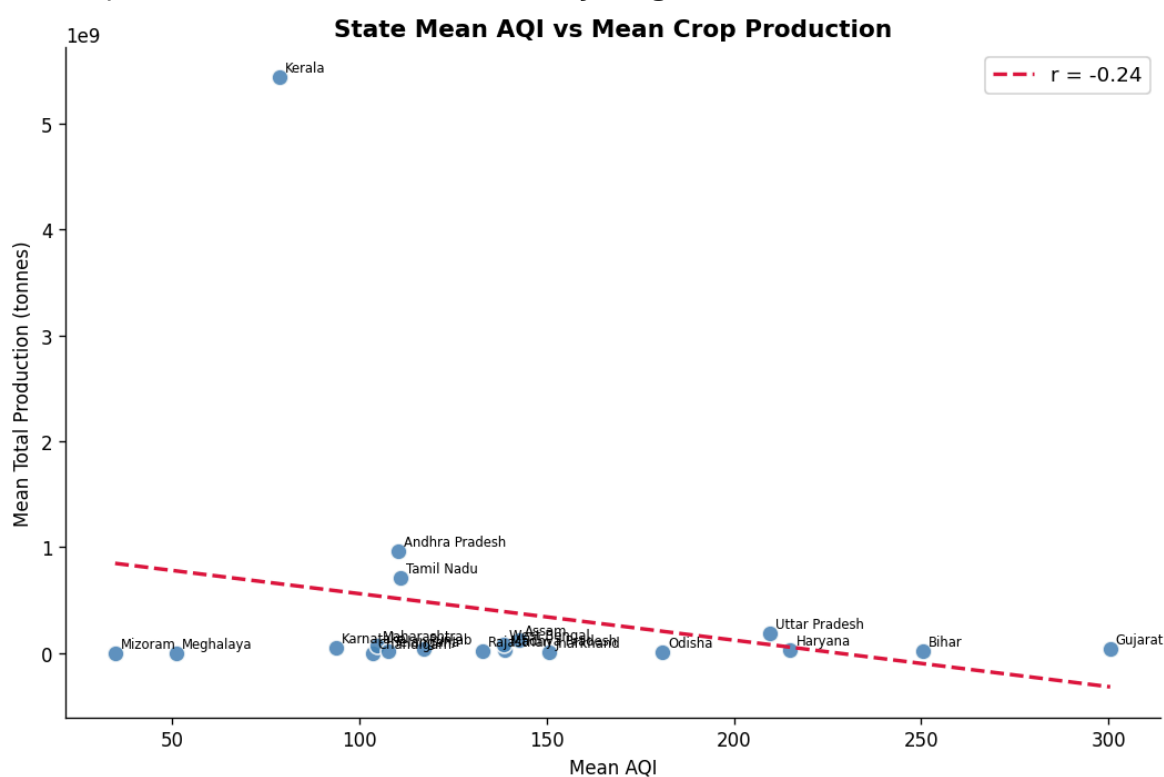
```
# Regression Line
m, b = np.polyfit(state_summary['Mean_AQI'], state_summary['Mean_
x_line = np.linspace(state_summary['Mean_AQI'].min(), state_summa
ax.plot(x_line, m * x_line + b, color='crimson', lw=2, linestyle=
ax.legend(fontsize=11)

ax.set_title('State Mean AQI vs Mean Crop Production', fontsize=1
ax.set_xlabel('Mean AQI')
ax.set_ylabel('Mean Total Production (tonnes)')
plt.tight_layout()
plt.savefig('taskB_scatter.png', bbox_inches='tight')
plt.show()
```

Pearson $r = -0.236$

p-value = 0.316

Interpretation: Not statistically significant at 95% confidence



Interpretation for the Research Team

A correlation value close to 0 indicates little to no linear relationship between state-level AQI and crop production in this dataset. This does **not** mean pollution has no effect on crops — it means this dataset, at this level of aggregation, cannot detect that effect. The data only overlaps for a single year (2015), limiting statistical power severely. A causal study would require multi-year panel data with crop-specific yields, meteorological controls, and irrigation records. The current dataset is insufficient to fund a causal claim.

Task C — One Plot to Rule Them All

Chosen visualisation: Task 7 — Monthly Median AQI across all cities (reproduced below)

```
In [46... fig, ax = plt.subplots(figsize=(13, 5))

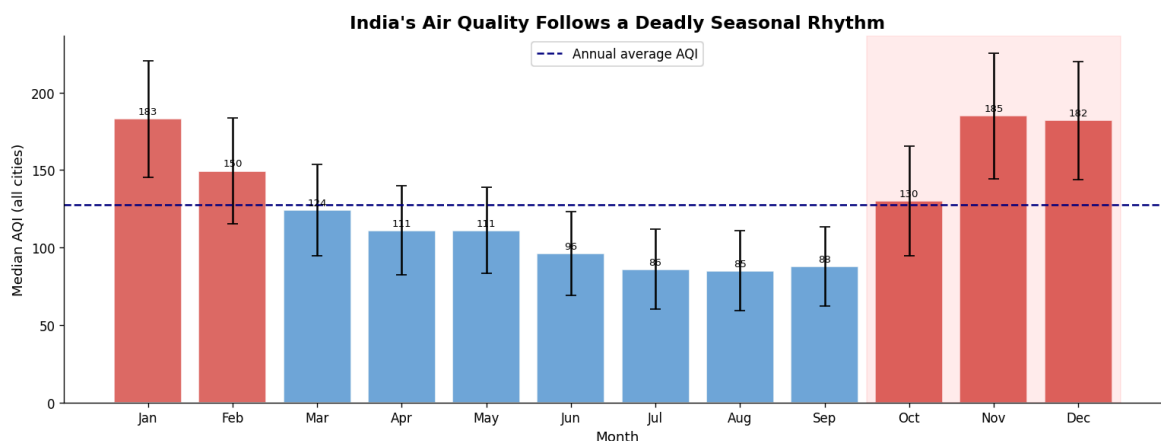
bar_colors = ['#d9534f' if m in [10, 11, 12, 1, 2] else '#5b9bd5']
bars = ax.bar(monthly['MonthName'], monthly['median'], color=bar_
               edgecolor='white', yerr=monthly['std']/3, capsiz=4

ax.axvspan(8.5, 11.5, alpha=0.07, color='red')
ax.axhline(monthly['median'].mean(), color='navy', lw=1.5, linestyle='dashed')

ax.set_title('India\'s Air Quality Follows a Deadly Seasonal Rhythm')
ax.set_xlabel('Month', fontsize=11)
ax.set_ylabel('Median AQI (all cities)', fontsize=11)
ax.legend(fontsize=10)

for bar, val in zip(bars, monthly['median']):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 1,
            f'{val:.0f}', ha='center', va='bottom', fontsize=8)

plt.tight_layout()
plt.savefig('taskC_hero_chart.png', bbox_inches='tight')
plt.show()
```



Caption

India's air quality follows a deadly seasonal rhythm that peaks exactly when farmers burn, and stays dangerously high all winter. Unlike any other visualisation in this analysis, this chart shows not just *that* pollution is severe, but *when* it strikes — revealing a five-month window (October to February) during which India's most vulnerable populations, including agricultural

workers, breathe air in the 'Poor' to 'Very Poor' range every single day. This seasonal structure is the single most actionable insight in the dataset: it tells a policymaker not just what to fix, but *when* to act.

Caveat: Correlation between harvest season and high AQI does not prove that burning is the sole cause. Temperature inversions, winter vehicle patterns, and reduced wind speeds are confounding factors this dataset cannot isolate.